1

PART 1: FOUNDATION

Specifications and Prompts

A developer at Riverside Clinics opens an AI assistant and types one line: "Build a feature that lets patients book appointments online." Ninety seconds later, she has working code. A booking form. A database table. Validation logic. A handful of passing tests. It looks finished.

She opens a pull request. Her team reads it and starts finding problems within the first few minutes. The booking form lets a patient pick any doctor at any clinic location, but Riverside's doctors only work specific locations on specific days, a rule everyone on the team knows and nobody wrote down. The appointment status field uses values the rest of the system doesn't recognize. The error handling doesn't match anything else in the codebase. None of this code is broken. It runs. It would even satisfy someone who had never seen Riverside's other systems. It is code built from a guess about how a clinic group probably works. Not for this one.

The Development Bottleneck

This scene repeats across organizations that adopt AI coding assistants expecting to move faster. Many of them do, for the first draft. Then a reviewer opens the pull request, and the hours saved writing the code get spent finding what it got wrong: the assumptions it made about location rules, status values, error conventions, none of them stated anywhere the AI could have read them.

The rewrite that follows takes hours of manual correction, sometimes days, done by the same developer the tool was supposed to speed up. Multiply that across a team shipping several features a week, and the productivity gain the tool promised doesn't show up in the sprint numbers. It shows up as review time, rework time, and a growing suspicion that the tool wasn't the upgrade it was sold as.

The tool isn't the problem. A traditional engineering team assumes the bottleneck is the person typing, and optimizes for that: faster typing, better frameworks, more automated tests. Much of the tooling built for developers has aimed at that assumption. AI coding assistants are the most extreme version of it, and they expose why the assumption was wrong. Developers were never the slow part. Figuring out what to build, whose rules applied, and how the new feature fit what already existed, that was always the slow part. It was just slow in a way that looked like typing, because writing the code was how a developer worked the problem out.

Take typing out of the loop entirely, hand it to a model that can produce a plausible booking form in ninety seconds, and the part that was never about typing is still there, waiting to be done by someone. The AI cannot do it for you, because it was never given the information that doing it requires.

Guessing Requests and Specification Requests

"Build a feature that lets patients book appointments" and "implement the appointment-booking use case" can look interchangeable. They are asking for two different kinds of work.

The first request describes an outcome and leaves every decision behind it unstated: which doctors are eligible, what counts as an available slot, what happens when two patients try to book the same slot at once, what the system calls a cancelled appointment versus a missed one. An AI assistant answering that request has to invent answers to all of it, because the request contains none of them. Most of the time, the invented answers are wrong. A guess about an organization it has never seen is usually a guess about the wrong organization.

The second kind of request names a decision that has already been made. Simon Martinelli, whose specification-driven methodology this book draws on throughout, puts the line there: the first request asks AI to reverse-engineer a business, the second asks it to translate a decision. A **specification**, the term this book uses for that decision written down, is a precise, technology-independent description of what a system must do: which actors are involved, what the rules are, what counts as success, and what counts as a failure the system has to handle. Writing one doesn't make the work disappear. It moves the work to where a human is still needed, deciding what the business actually requires, and away from where AI now does it faster than any developer can type: translating an already-made decision into working code.

> **Ask AI to guess your business, and it will. Ask it to implement a decision your team has already made, and it will do that instead.**

Identifying Specification Requests

Before asking an AI assistant to build anything, check which kind of request you're about to make. The test takes longer to explain than to run.

1. Write down the request exactly as you'd type it to the AI assistant, in full, before you send it.
2. Check it for three things: named actors (which users or roles, specifically), the data involved and any constraints on it, and at least one business rule that already exists somewhere, even if only in someone's head.
3. If the request names a feature but two or more of the three are missing, stop. Write down the missing decision first. A feature name is a label for the specification you haven't written yet.

Skip this test for a throwaway script or a small one-off change; neither needs it. Its job is to catch one moment: the point where a request about to go out is really a request for the AI to guess, before code comes back that nobody wanted.

Worked Example: Riverside's Booking Request

Here is the Riverside Clinics request from the start of this chapter, run through the test, next to the version the rest of this book builds toward.

Guessing-Shaped Request

"Build a feature that lets patients book appointments online."

No actor named beyond "patients." No data constraints. No business rule. Every decision, which doctors, which locations, what counts as available, is left for the AI to invent.

Specification-Shaped Request

"Implement the Schedule Appointment use case (Chapter 3) against the Riverside Clinics domain model (Chapter 4): a patient books one open slot with one eligible doctor at one location, subject to the 90-day booking window agreed with clinic operations, following the error-handling and validation conventions in the Spring Boot skill (Chapter 7)."

Every decision the first version left open is either answered or points to where the answer already lives.

The rest of this book is spent building the documents that second request depends on: a stated reason the feature exists, requirements precise enough to test, a domain model of patients and doctors and appointments, and the architectural and implementation conventions that keep this feature consistent with everything else Riverside has built.

Limitations

▲ WARNING

The test in this chapter flags an obviously under-specified request. It does not guarantee a good specification. A request can name actors, data, and a rule, and still be wrong, if the underlying business decision was never actually settled, only assumed. Writing it down doesn't fix a decision nobody made correctly in the first place; it just makes the mistake visible sooner, and in front of the right people.

Treating every request as needing a full specification	A team spends a day documenting a five-line configuration change, and starts resenting the process instead of the rework it was meant to prevent.
Treating a specification-shaped request as automatically correct	A precisely worded request built on a wrong assumption produces precisely wrong code, faster than a vague one would have.

Checklist: Specification-Shaped Requests

Actors	Guessing-shaped: "patients," "customers," nobody in particular. Specification-shaped: which users, in which role, named as specifically as the business names them.
Data and constraints	Guessing-shaped: no constraint on anything. Specification-shaped: the data involved and the limit that applies to it, such as Riverside's 90-day booking window.
Existing business rule	Guessing-shaped: every rule left for the AI to invent. Specification-shaped: at least one rule that already exists somewhere, even if only agreed verbally.

If a marker is missing, there is one more question to ask: is there a person who can supply it before the request goes to AI, rather than after the code comes back?

Run this before sending a feature request to an AI assistant. If two or more of these come back on the guessing side, the request is asking AI to guess.

Summary

- The bottleneck in software delivery was never typing speed. It was always understanding what to build; AI has only made that gap visible by removing the typing.
- A request that names only a feature asks AI to invent the business decisions behind it. A request that names actors, data, and rules asks AI to implement decisions a team already made.
- Code that runs correctly can still be wrong for a project, if it wasn't built from that project's own decisions.
- Writing a specification doesn't remove the hard part of the work. It moves it earlier, to the person who understands the business, and away from the part AI now does faster than any developer types.

DISCUSSION QUESTIONS

- The last time your team rejected AI-generated code in review, what specifically was wrong with it?
- How many of the feature requests your team gave an AI assistant this month would pass the test in this chapter?

PRACTICAL EXERCISE

Pull up the last feature request your team gave an AI assistant. Run it through this chapter's checklist before you ask the assistant for anything else.

Specifications and Prompts ends here.